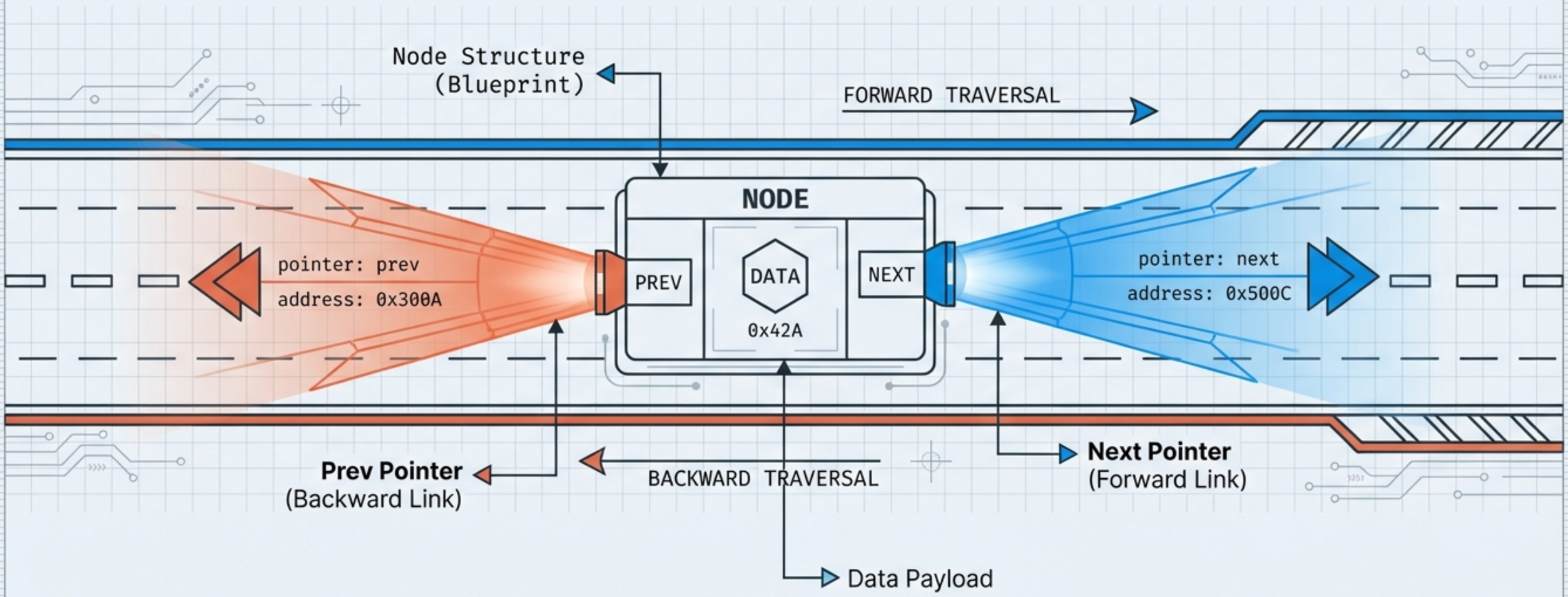
Beyond the One-Way Street

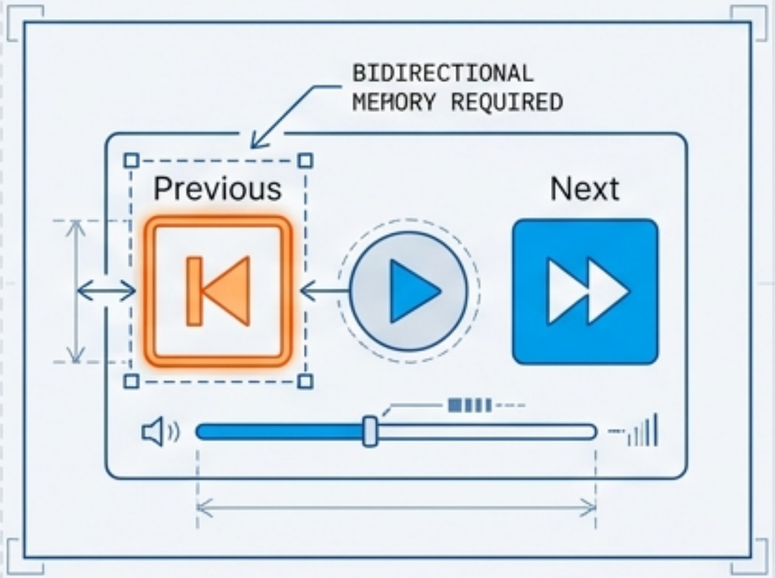
Mastering the mechanics of Doubly Linked Lists & Circular Linked Lists



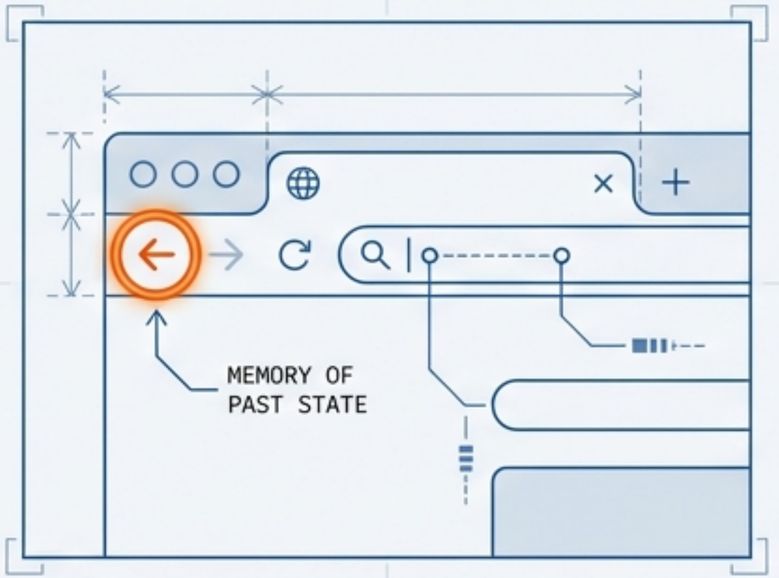
Based on 'Chai aur Code: Doubly Linked List and Circular Linked List' | Data Structures & Algorithms Series

When You Need to Look Back

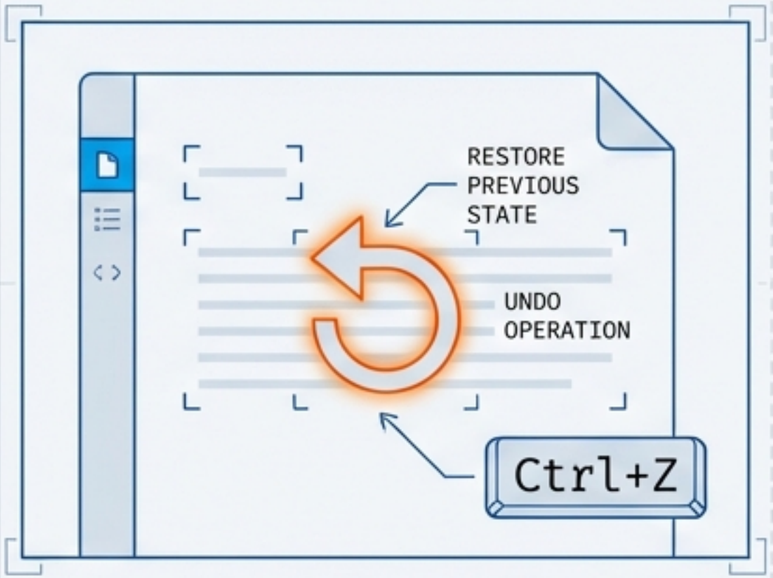
Singly Linked Lists have a critical blind spot: movement is unidirectional. Once a node is passed, it is lost unless traversal restarts. Real-world systems require bidirectional memory.



Music Player: Previous Track



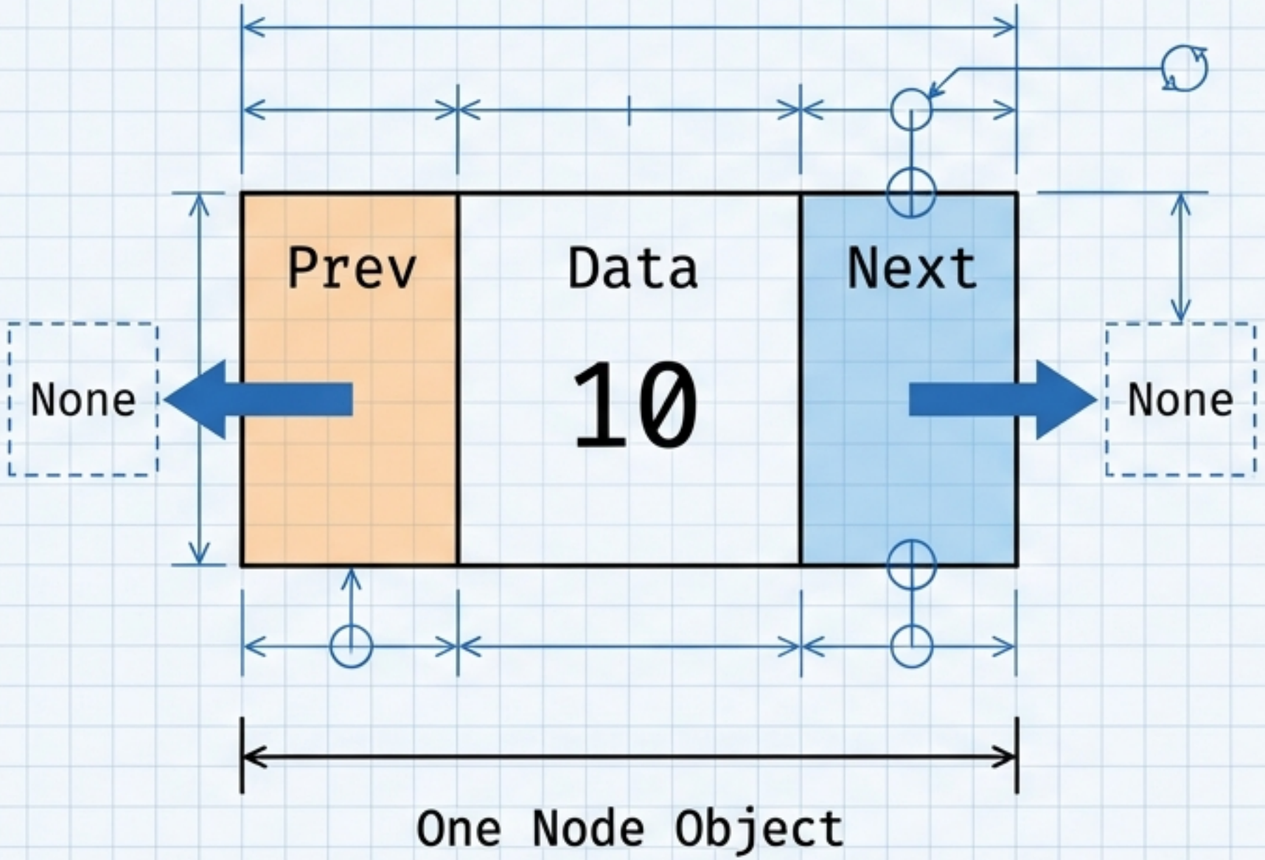
Browser History: Back Button



Text Editors: Undo (Ctrl+Z)

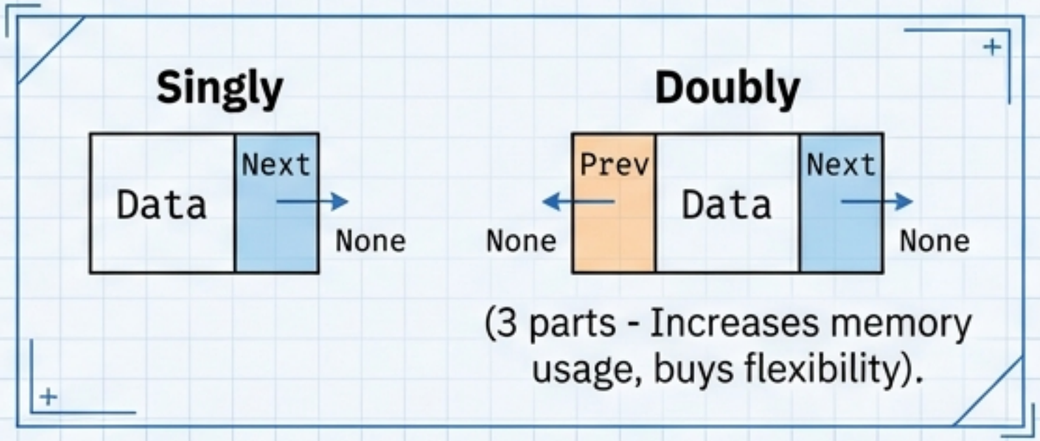
The Solution: A **Doubly Linked List** (DLL) gives every node a memory of both its future (**Next**) and its past (**Prev**).

The Anatomy of a DLL Node



The Blueprint (Python)

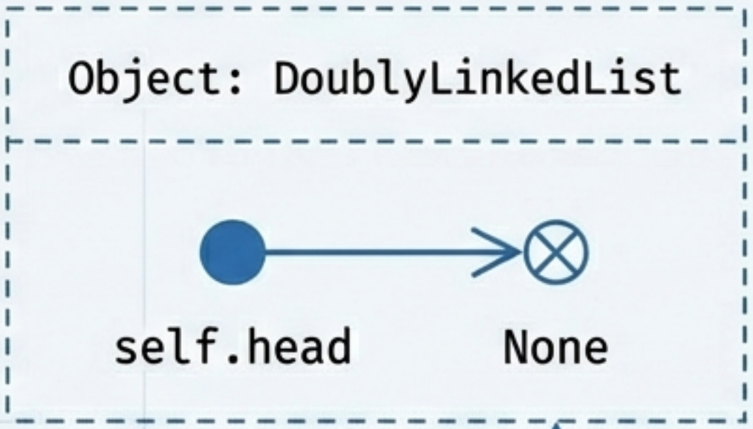
```
class Node:
    def __init__(self, value):
        self.data = value
        self.next = None
        self.prev = None  # The Upgrade
```



Initializing the System

CONCEPT

- We separate the “Node” (the brick) from the “Linked List” (the wall).
- The **DoublyLinkedList** class acts as the controller. It holds the primary entry point: the Head.
- **State**: An empty list means Head is None.



VISUAL & CODE

```
class DoublyLinkedList:
    def __init__(self):
        self.head = None
```

Navigation and Boundaries

Going Forward

```
while t.next is not None:
    t = t.next
```

Stop when `t.next` is None (Tail)

Going Backward

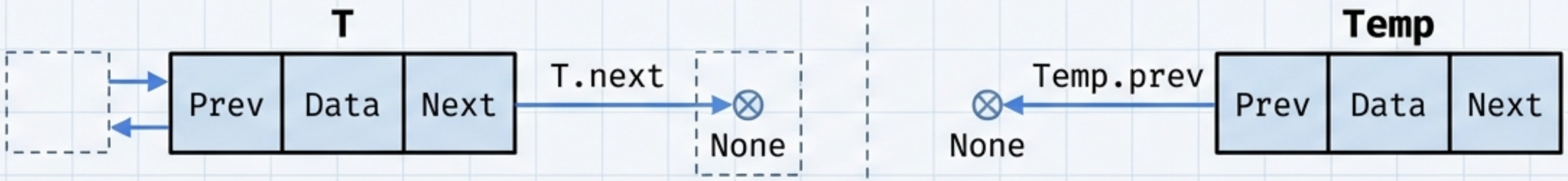
```
while t.prev is not None:
    t = t.prev
```

Stop when `t.prev` is None (Head)

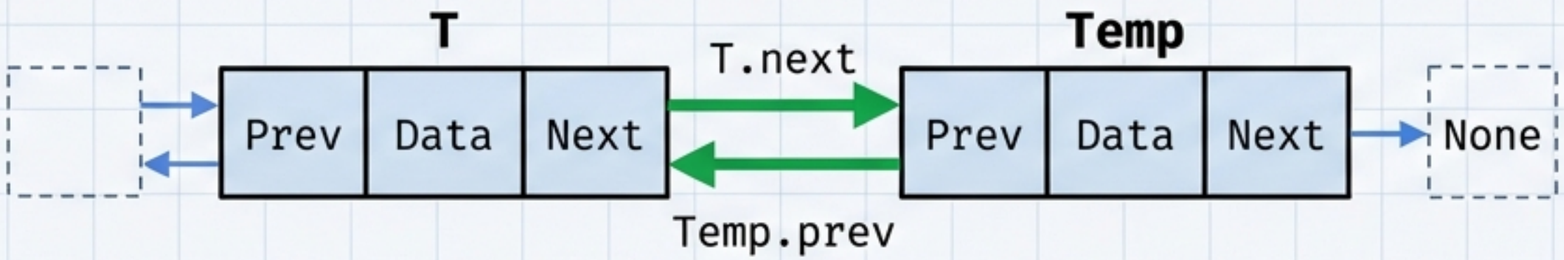
Insertion at the End (Append)

⚠ **Edge Case:** If Head is None, then Head = Temp.

BEFORE (Current State)



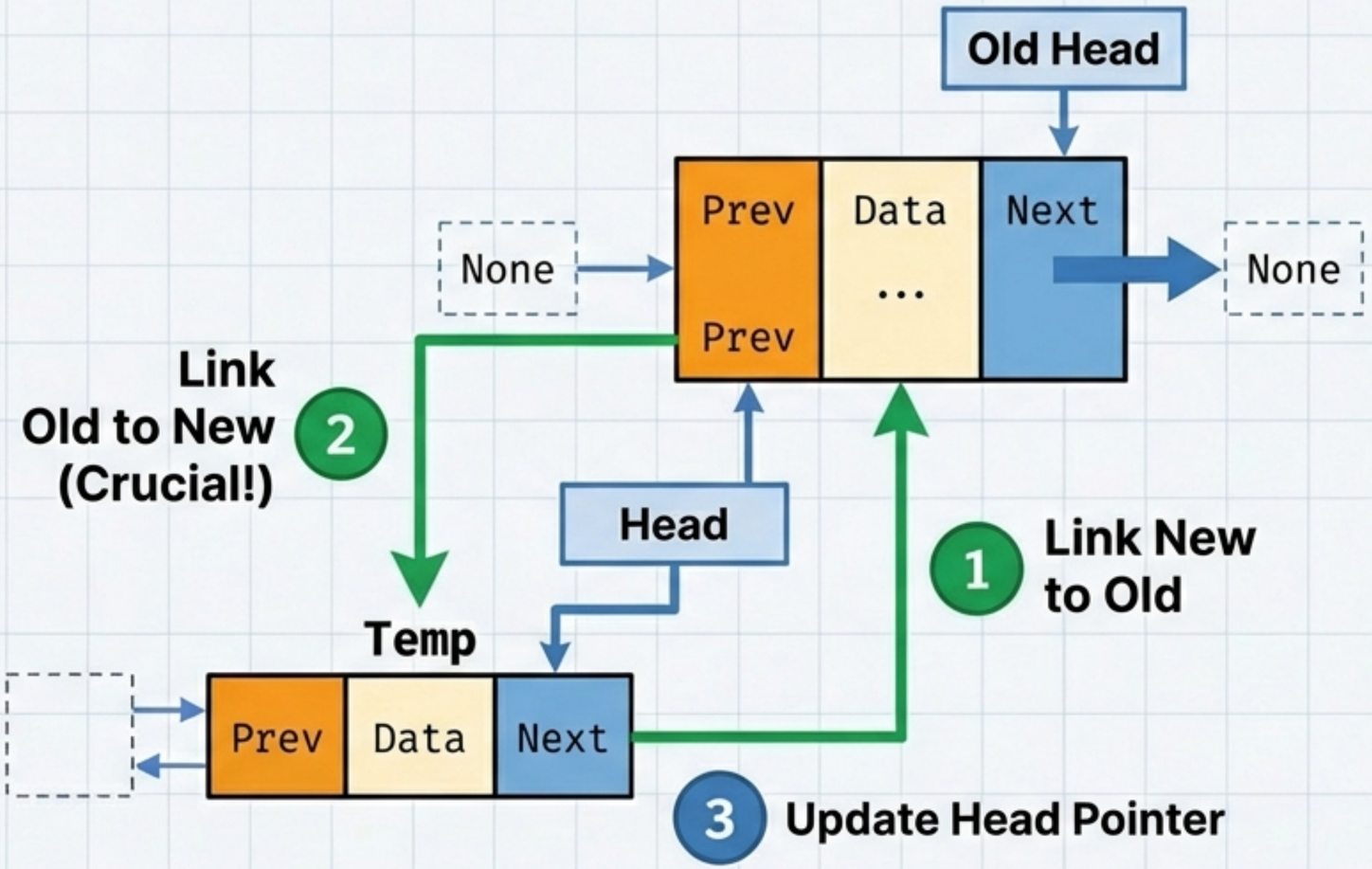
AFTER (The Logic)



Code Logic

1. Traverse to Tail (T)
2. T.next = Temp
3. Temp.prev = T

Insertion at the Beginning (Prepend)

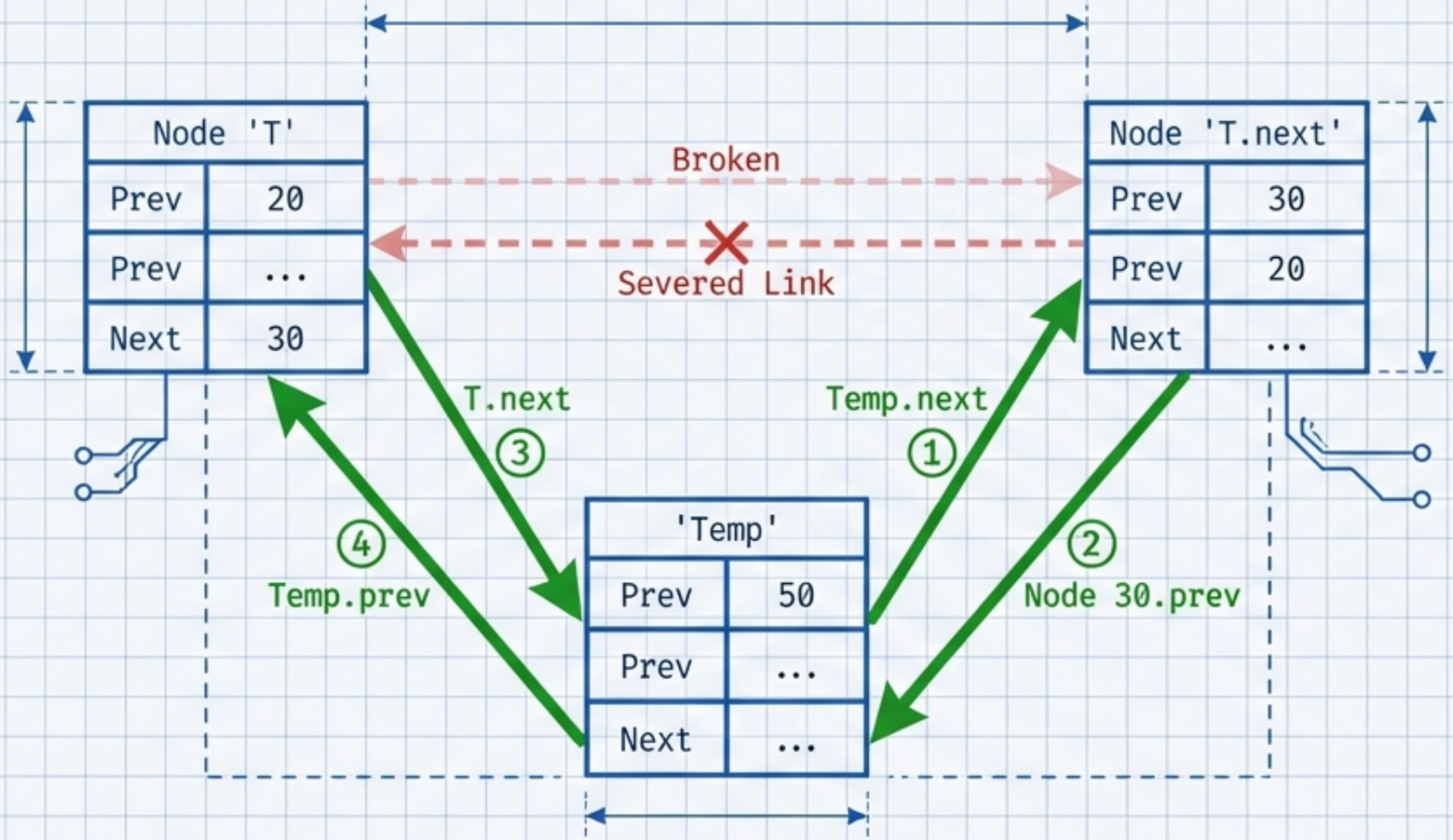



```
if self.head is None:
    self.head = temp
else:
    temp.next = self.head
    self.head.prev = temp
    self.head = temp
```

Warning: Never move the Head pointer before establishing the links, or the list is lost.

The Surgical Insert (Middle)

Goal: Insert Node (50) between Node (20) and Node (30).

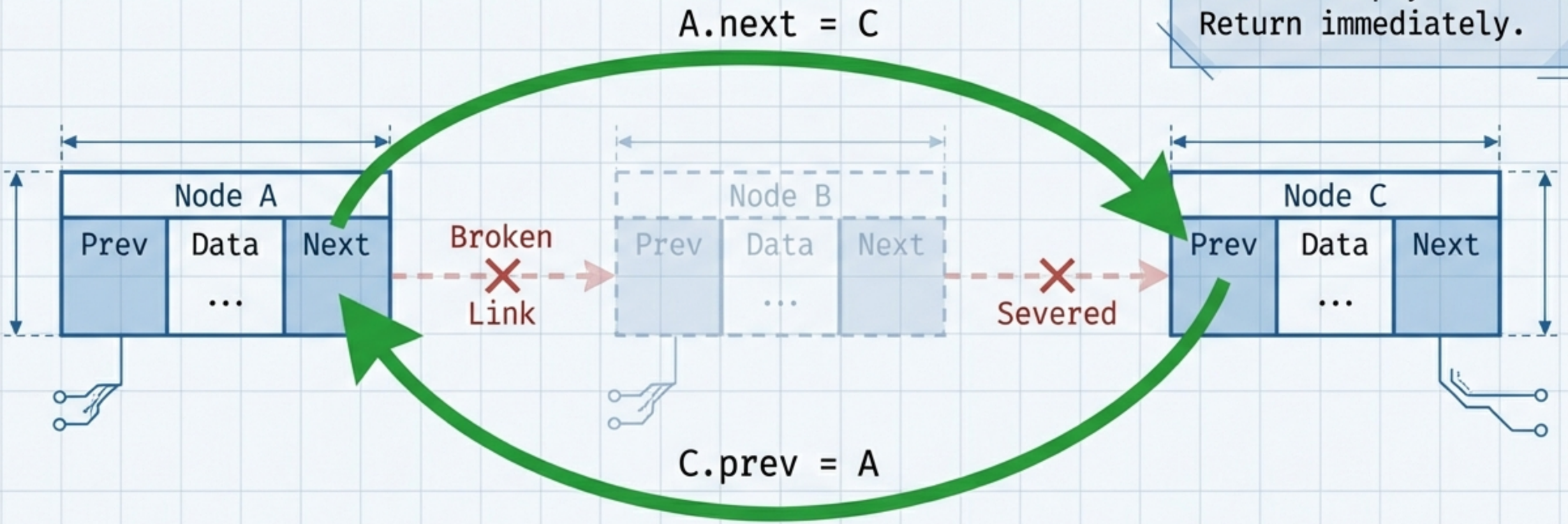


Code Logic

1. Connect Temp forward to 30.
2. Connect 30 backward to Temp.
3. Connect 20 forward to Temp.
4. Connect Temp backward to 20.

Deletion Strategy: The Algorithm

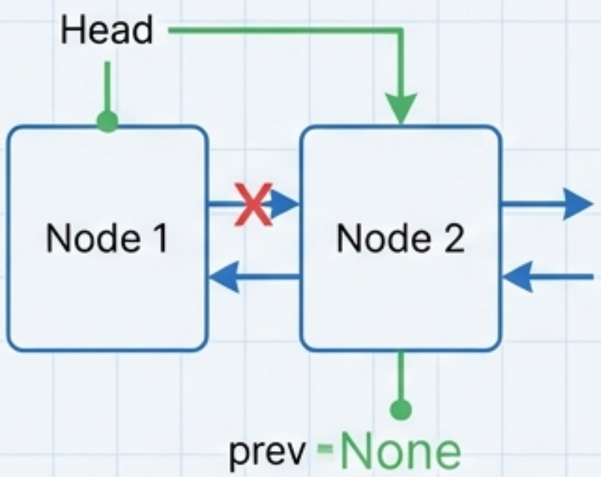
Underflow Check:
If Head is None,
list is empty.
Return immediately.



1. **Search:** Traverse to find the target node (T).
2. **Identify:** Check position (Head? Tail? Middle?).
3. **Re-link:** Update neighbors (A and C) to hold hands, dropping B.

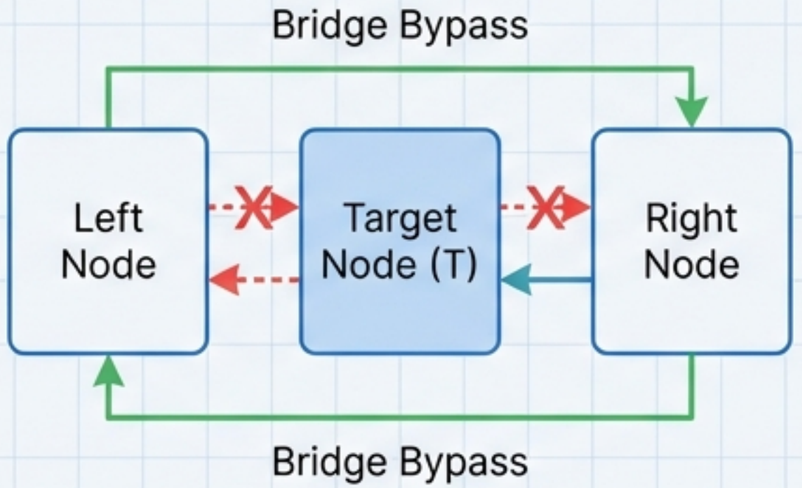
Deletion Mechanics & Edge Cases

Delete Head



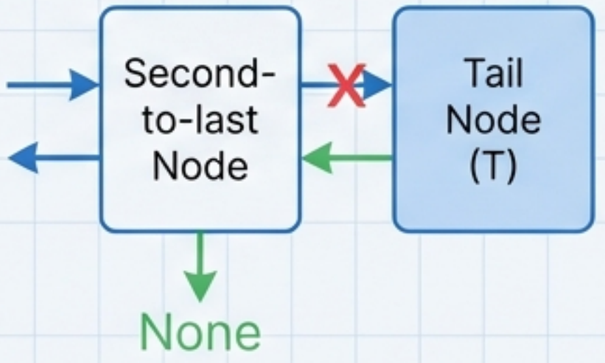

```
head = head.next
head.prev = None
```

Delete Middle




```
t.prev.next = t.next
t.next.prev = t.prev
```

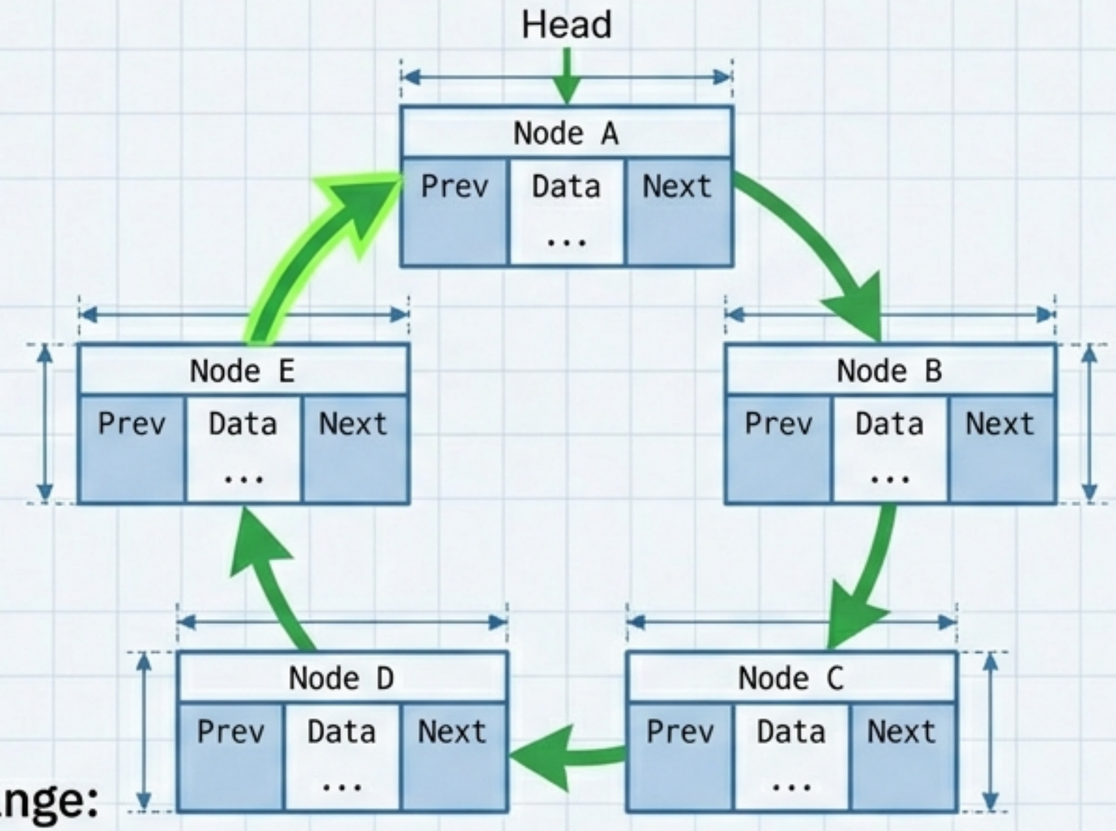
Delete Tail




```
t.prev.next = None
```

Unified logic handles clean-up across the entire lifecycle of the list.

The Infinite Loop: Circular Linked Lists



Singly Linked List:
`Tail.next → None`

Circular Linked List:
`Tail.next → Head`

Traversal Condition Change:

```
while t.next != self.head:
```

(Instead of `!= None`)

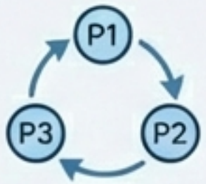
Warning: Standard loops will cycle forever (Infinite Loop).

Why Build a Circle?

Real-world applications of cyclic data structures.

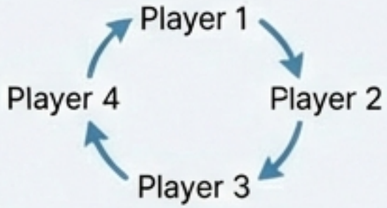
Round Robin

Time-slicing processes. P1 → P2 → P3 → P1. The list never ends, it just cycles.



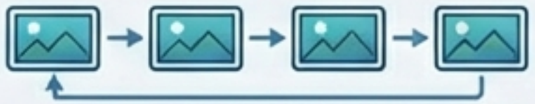
Turn Management

Multiplayer turns. Player 4's turn ends, control passes instantly back to Player 1.

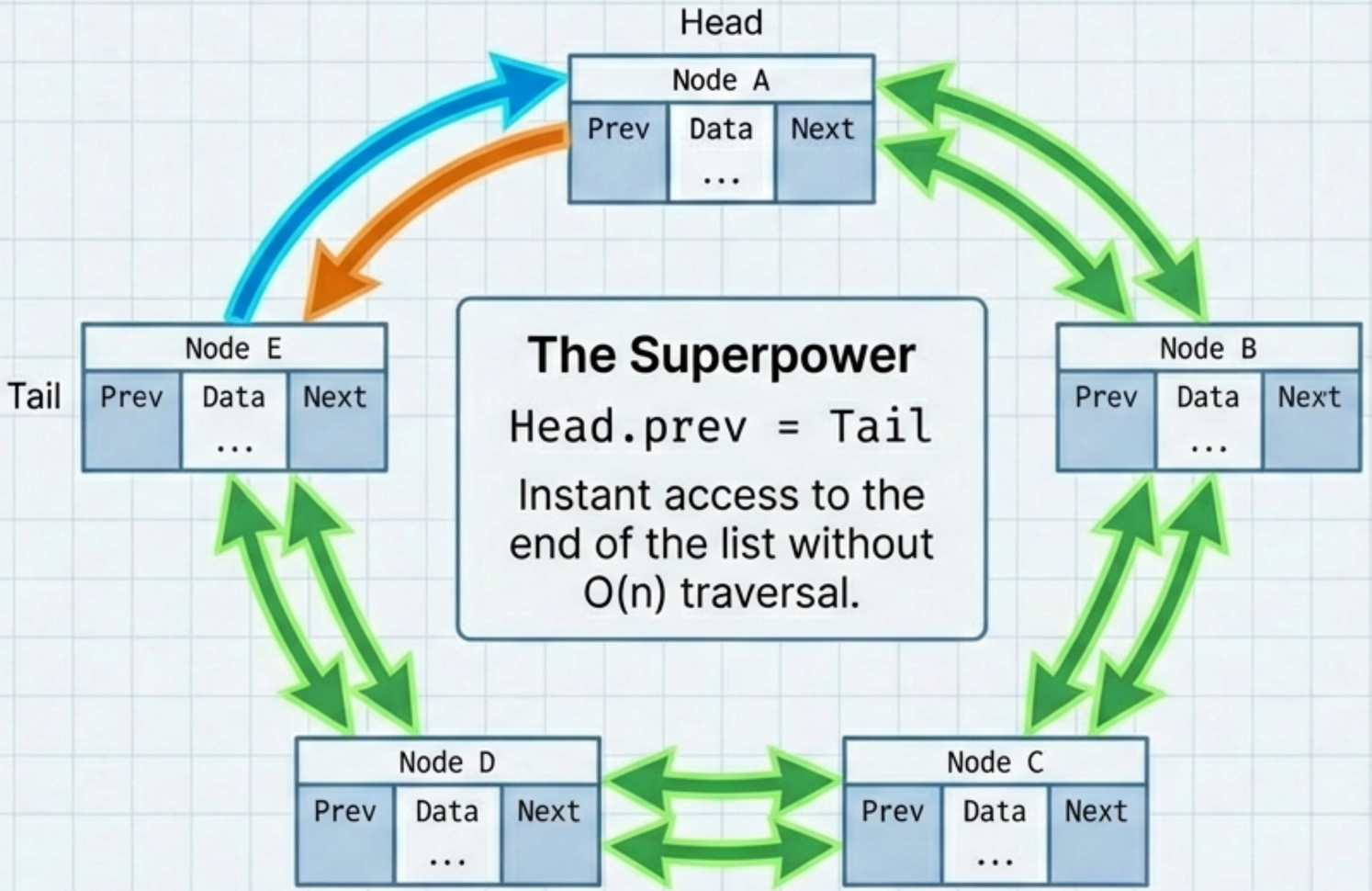


Infinite Scroll

Clicking 'Next' on the last image seamlessly loops to the first image.



The Hybrid: Circular Doubly Linked List



Choosing Your Structure

Structure	Pros	Cons	Best Use Case
Singly Linked List	Low Memory, Simple	No Backward Traversal	Simple Stacks, Forward Streams
Doubly Linked List	Bidirectional, Flexible Deletion	Extra Memory (Prev ptr), Complex Code	Music Players, Browser History
Circular List	Infinite Cycling	Infinite Loop Risk	Round Robin, Multiplayer Games

The Challenge

Understanding the logic is step one.
Writing the code is step two.

```
/*
  HOMEWORK:
  1. Define the Class.
  2. Handle the edge case: The first node must point to itself (Next & Prev).
  3. Share your code.
*/
```

Don’t just memorize the code. Visualize the pointers.
If you can draw it, you can code it.